CS 122A Custom Laboratory Project Report

Pico/FPGA RC Signal Converter and Dashboard

Danny Topete, Troy Drescher

High-level description of the project

Our project implements a multi-stage remote-control signal converter and live dashboard, using a Raspberry Pi Pico 2W to receive ExpressLRS (ELRS) CRSF telemetry and display channel values on an FPGA-driven 4.3″ RGB565 LCD. The Pico parses incoming RC channel data over UART, converts it into a format usable for visualization, and streams framebuffer updates to an iCE Sugar Pro FPGA display controller.

Elements of complexity

- Real-time parsing of high-speed ELRS CRSF packets at 420000 baud using the Raspberry Pi Pico UART.
- Integration of LVGL on the Pico to render a real-time channel monitor UI with eight channels, including dynamic bar graphs and numeric labels.
- SPI-based framebuffer output from the Pico to the FPGA ribbon cable display.
- Hardware button interrupt-driven redraw requests to redraw the screen when display artifacts occur.
- Developed a custom ExpressLRS firmware for the ESP32-S3 to receive CRSF telemetry data.
- Pico-side PPM signal generation from decoded CRSF channels.
- FPGA-side HDL logic for PPM pulse measurement and two PWM outputs.

User guide: explain how the user will interact with the system

1. Power the Pico and FPGA display hardware.
2. Pair a remote controller to the ESP32-S3 LoRA receiver to start receiving CRSF packets.
3. The FPGA's LCD shows eight channel bars and numerical values representing RC channel positions inputs from the remote.
4. If the display fails to render properly (i.e. random artifacts), press the GP15 button to force a redraw the screen.
5. To inspect serial telemetry, connect to the Pico USB serial console and observe `UART1/GP5` status output.
6. The Pico also generates a PPM signal from the decoded channels for the FPGA-side PWM output stage.
7. Either using a second Pico or the FPGA connect the Motor driver to the FPGA or pico PWM output.
8. Now the Motor driver will drive the motors based on the PWM signal generated from the decoded PPM.

List of hardware components used

- Raspberry Pi Pico 2W
- iCE Sugar Pro FPGA with development board
- 4.3″ TFT LCD 480x272 RGB565 display with PMOD interface
- ESP32-S3 with integrated LR1121 module LoRA development board running custom ExpressLRS firmware
- RadioMaster Zorro running EdgeTX with Ranger Nano ELRS Transmitter Module running customized ExpressLRS firmware.
- YoungRC Drive ESC Brushed Electric Speed Controller
- 3V-12V 1000RPM n20 brushed motor.

List of any software libraries used

- Raspberry Pi Pico C/C++ SDK
- LVGL graphics library
- Custom SPIDisplay framebuffer driver provided by Dr. Allan Knight
- CRSFReader protocol parser
- Pico `pico_stdlib`, `hardware_spi`, `hardware_uart`, and `hardware_gpio`
- Modified Custom version of ExpressLRS firmware. https://github.com/ExpressLRS/ExpressLRS

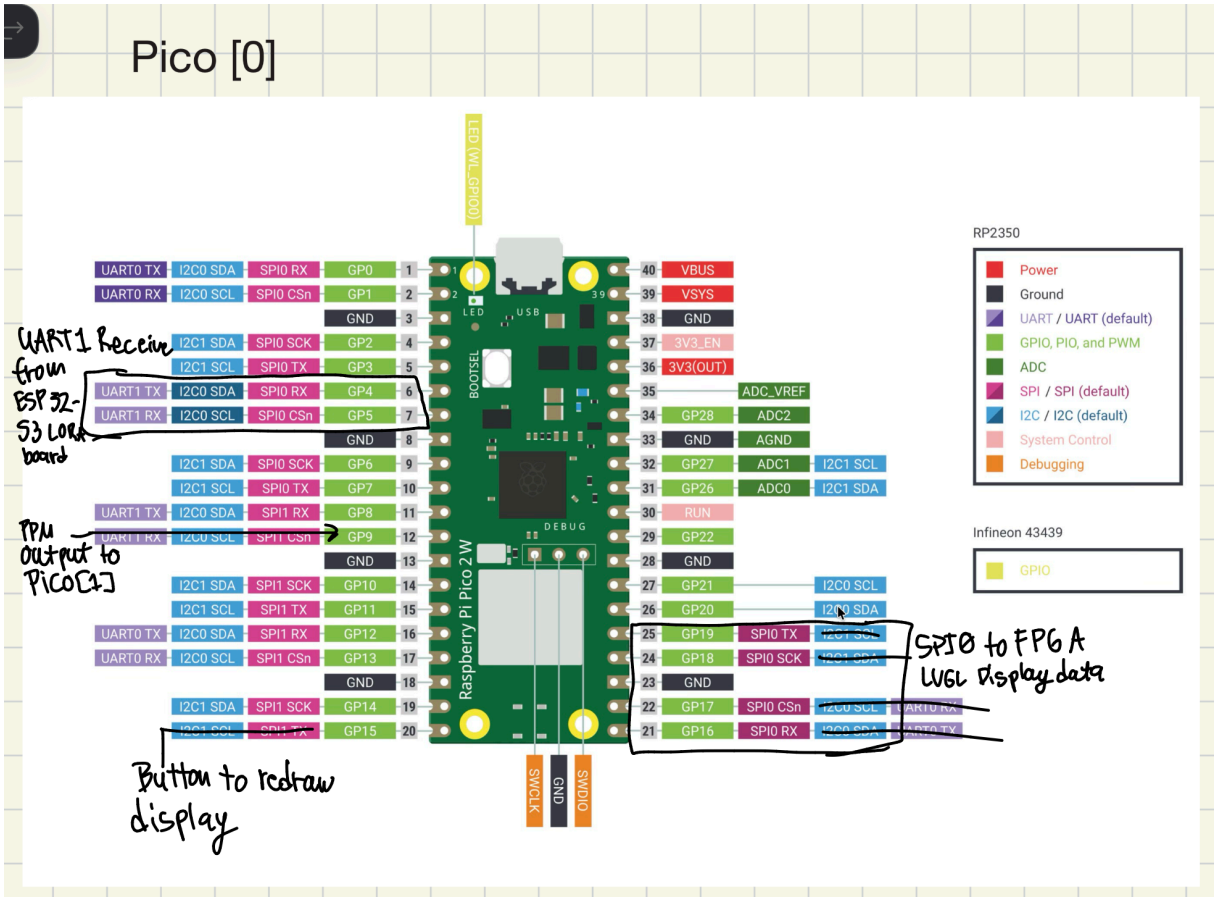
List of any protocols used

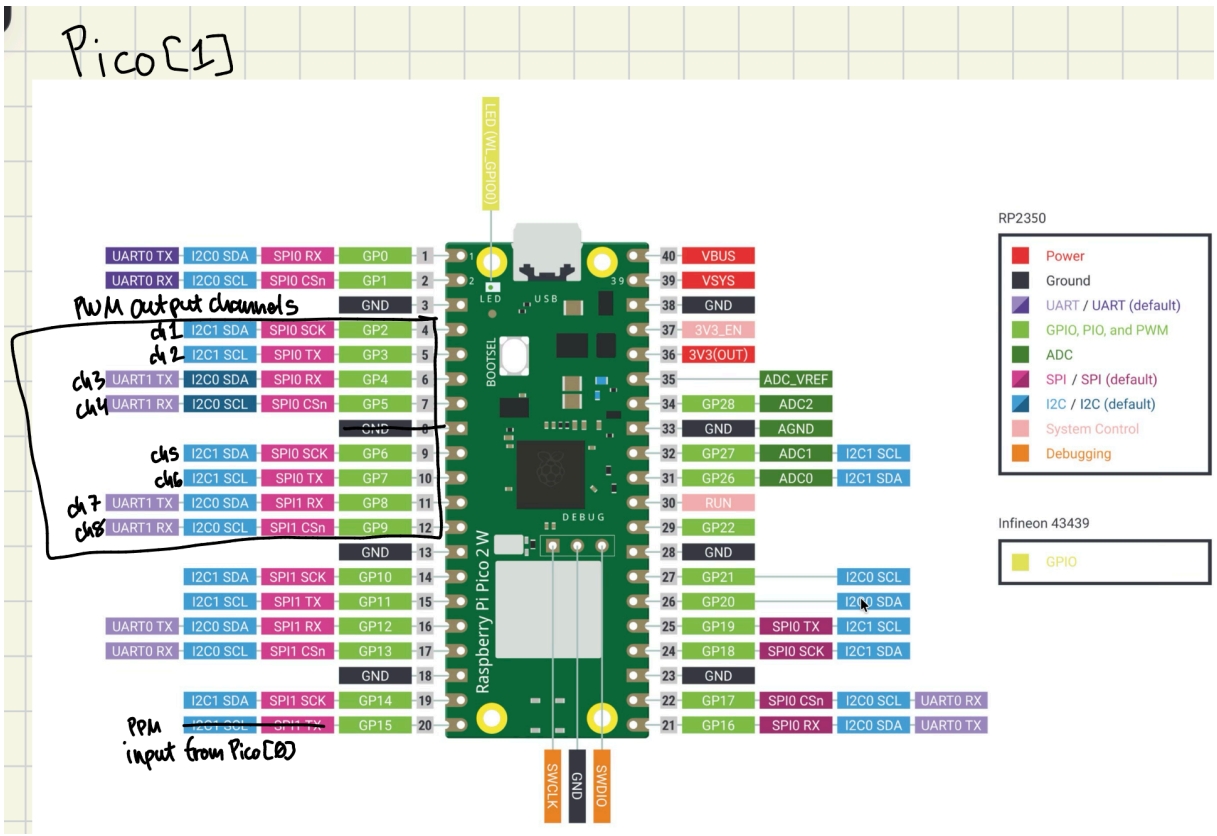
- ExpressLRS CRSF for RC channel telemetry input
- UART serial communication between ESP32 and Raspberry Pi Pico to transfer ExpressLRS(ELRS) data via Crossfire Serial Protocol (CRSF)
- SPI framebuffer transfer from Pico to FPGA to display
- LVGL internal display flush callbacks for screen updates

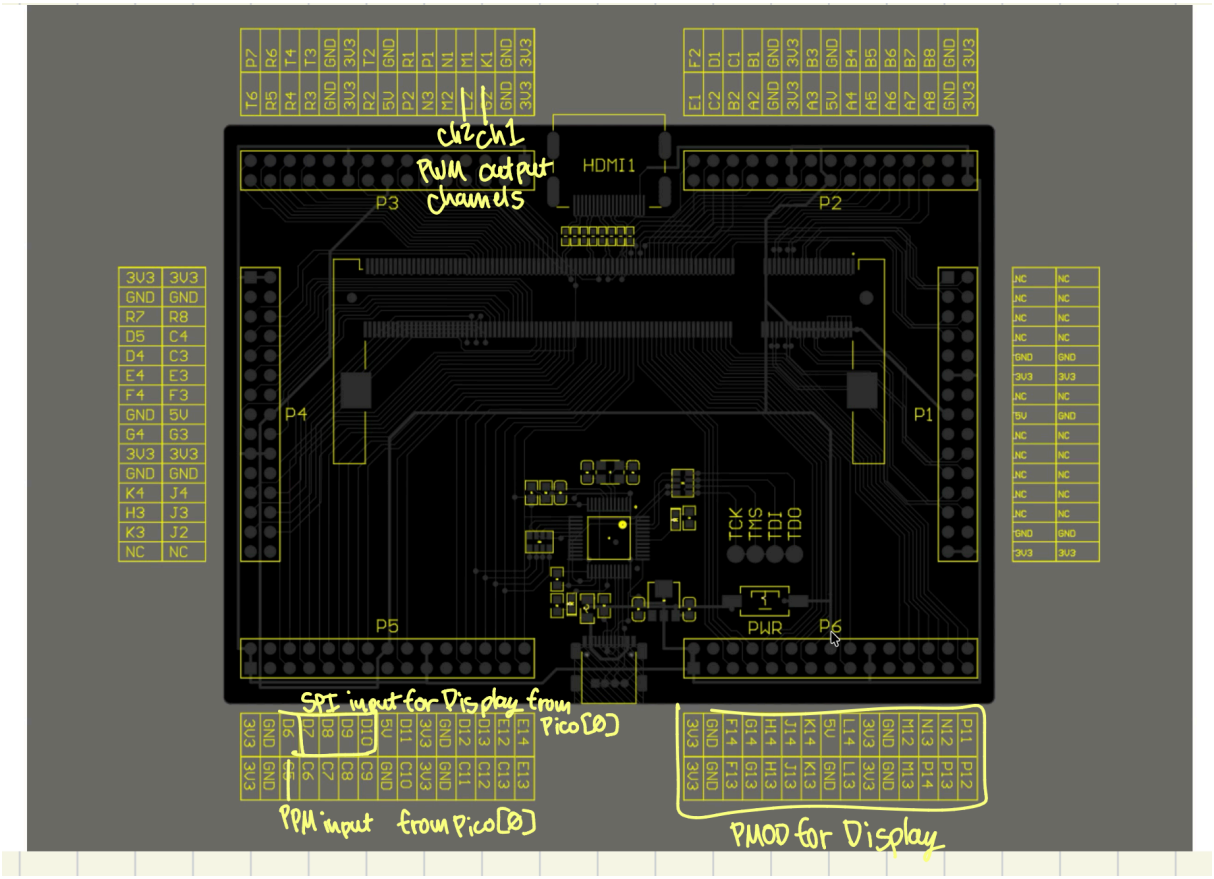
How you met the requirements listed in the proposal

We started by building a custom firmware for the ESP32-S3 LoRA to receive ELRS CRSF data from the remote controller. We then implemented the signal conversion pipeline by receiving ELRS CRSF on the Pico UART1 interface and parsing channel data with `CRSFReader`. The Pico renders an LVGL-based dashboard showing eight RC channels on the FPGA-driven LCD, meeting the display and monitoring requirements. The main Pico encodes the incoming data into PPM and outputs it to another Pico and the FPGA. The FPGA and the accesory Pico both decode the PPM signal into 8 PWM Channels. The system also supports a hardware button to request display redraws, and it reports serial monitoring status over USB serial. This satisfies the proposal's goals for real-time RC signal conversion, FPGA display integration, and user feedback.

Wiring diagram for the physical hardware setup





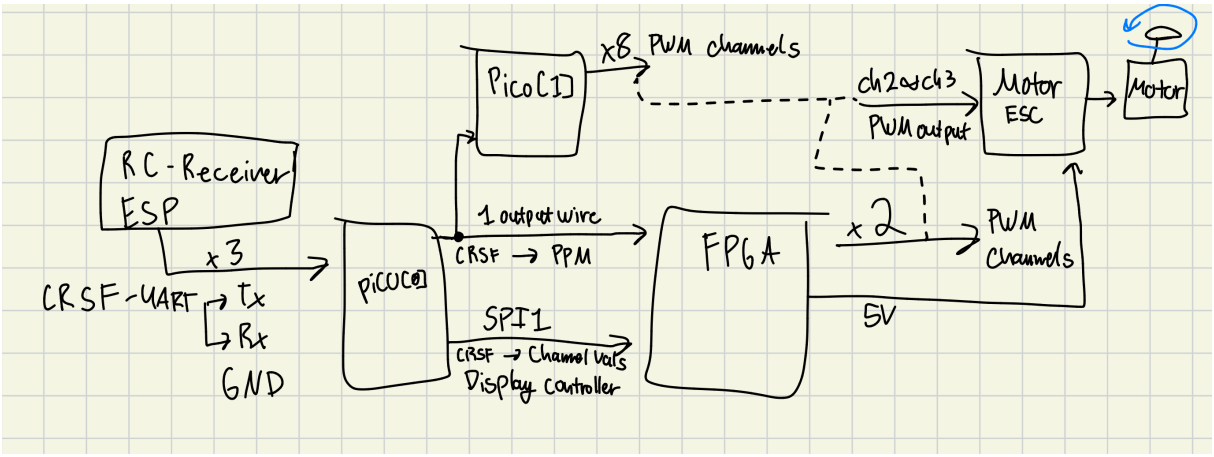


- Pico UART1 RX -> ELRS receiver TX / ESP32-S3 UART output
- Pico SPI0 MOSI/SCLK/CS -> FPGA display controller input
- Pico GP15 -> button input with pull-up to request display redraw
- 4.3″ TFT LCD powered from the FPGA board and driven by the FPGA framebuffer
- Pico USB -> host PC for power and serial logging

Design Diagram

The design consists of three main subsystems:

1. ELRS receiver input: ESP32-S3 sends CRSF data to the Pico UART1 RX pin.
2. Pico signal processor: `main.cpp` polls UART1, parses CRSF frames, updates channel values, and maintains an LVGL dashboard.
3. FPGA display output: the Pico sends framebuffer data over SPI to the iCE Sugar Pro board, which drives the 480x272 LCD.



AI usage

Limited AI assistance was used as a debugging aid during development, mainly for the custom ExpressLRS firmware and receiver configuration, and for getting the Pico CRSF UART input working by checking that CRSF data was being received correctly. Although, pyserial and printf statements were mainly used for debugging the data (PPM, PWM, and UART) sent and received across both Picos.

Acknowledgements

We thank the UCR CS122A course staff for the framebuffer and LVGL starter code, and the open-source communities behind the Raspberry Pi Pico SDK, LVGL, and ExpressLRS. Additional thanks to team member Troy Drescher for the custom ELRS firmware and receiver configuration work.